EXPERIMENT REPORT / PREPARED 13 NOVEMBER 2026

Revealing hidden portfolio concentration

TraderX + PostgreSQL + FalkorDB

A reproducible comparison of relational and graph traversal inside an open source trading application. Measurements were collected on 12 November 2026; this report summarizes the current local implementation and validation.

Why TraderX

TraderX is FINOS's Apache-2.0-licensed reference trading application. Accounts, orders, trades, positions, pricing and a trading interface provide a recognizable business workflow. Its specification-driven states make it possible to introduce a capability, change its implementation, and compare the outcome on repeatable datasets.

The question

"What do I really own?" Direct holdings can hide repeated exposure to the same corporate group through funds, funds of funds and subsidiary issuers. The experiment calculates that exposure and explains each contributing path.

Measured finding	Interpretation
Simple query: 2 paths	PostgreSQL median 0.324 ms; FalkorDB 0.526 ms. PostgreSQL is faster for this query.
Overlapping funds: 25 paths	PostgreSQL median 8.718 ms; FalkorDB 1.761 ms. FalkorDB is about 5× faster for this query.
Initial projection creation	PostgreSQL 0.781 s; FalkorDB 3.024 s. Faster graph traversal comes with greater projection setup work in this implementation.

The business result

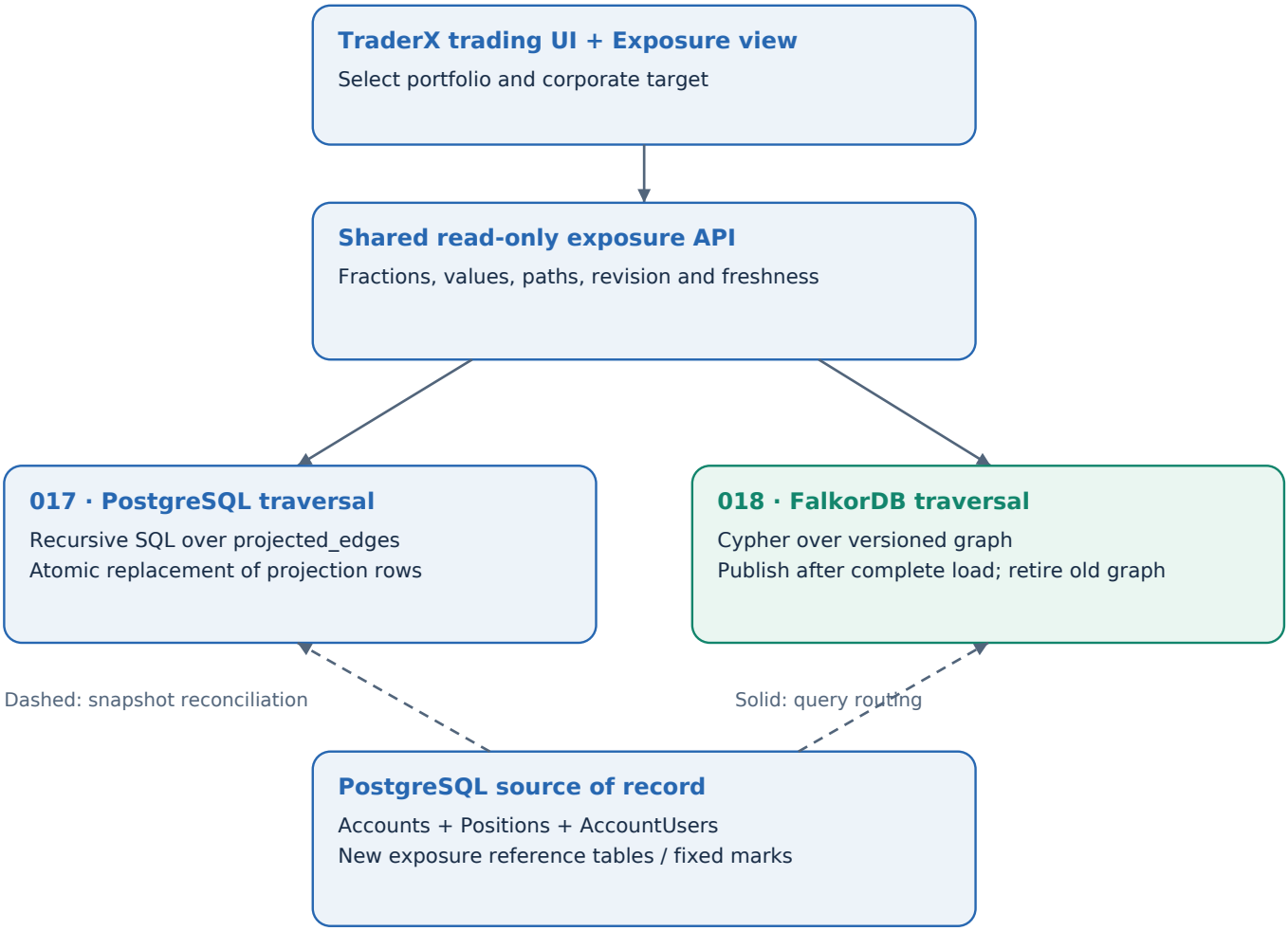
A known $100,000 synthetic portfolio shows $2,000 of direct corporate exposure and $12,000 through nested funds: **2% + 12% = 14%**. Both implementations return matching results. Increasing the direct holding by $2,000 changes exposure to $16,000 / $102,000, or approximately 15.6863%.

These are local microbenchmarks on synthetic data. They establish working behavior and a measurable traversal difference for the selected queries, not a general database ranking or production capacity guarantee.

01 / IMPLEMENTATION

How the experiment works

State **017-portfolio-lookthrough** builds on upstream **009-order-management-matcher (C2)**. State **018-falkordb-lookthrough** inherits 017 and changes the exposure-query backend. Both expose the same API and explanation view.



One source, two query implementations

• **Seed once:** load deterministic reference relationships and synthetic holdings into an isolated demo database; retain later position changes across restarts.

• **Reconcile:** read nodes, links and current positions in one repeatable-read snapshot. Validate long-only holdings, known marks, fund weights, acyclicity and depth; normalize positions into portfolio fractions.

• **Publish:** transactionally replace SQL projection rows, or build a complete versioned FalkorDB graph before switching the service pointer. Polling waits two seconds between refresh attempts; rebuild time adds to freshness latency.

• **Explain:** return direct, indirect and total exposure, portfolio/exposure values, contributing paths, valuation time, revision and projection status. Failed or stale projections reject successful reads.

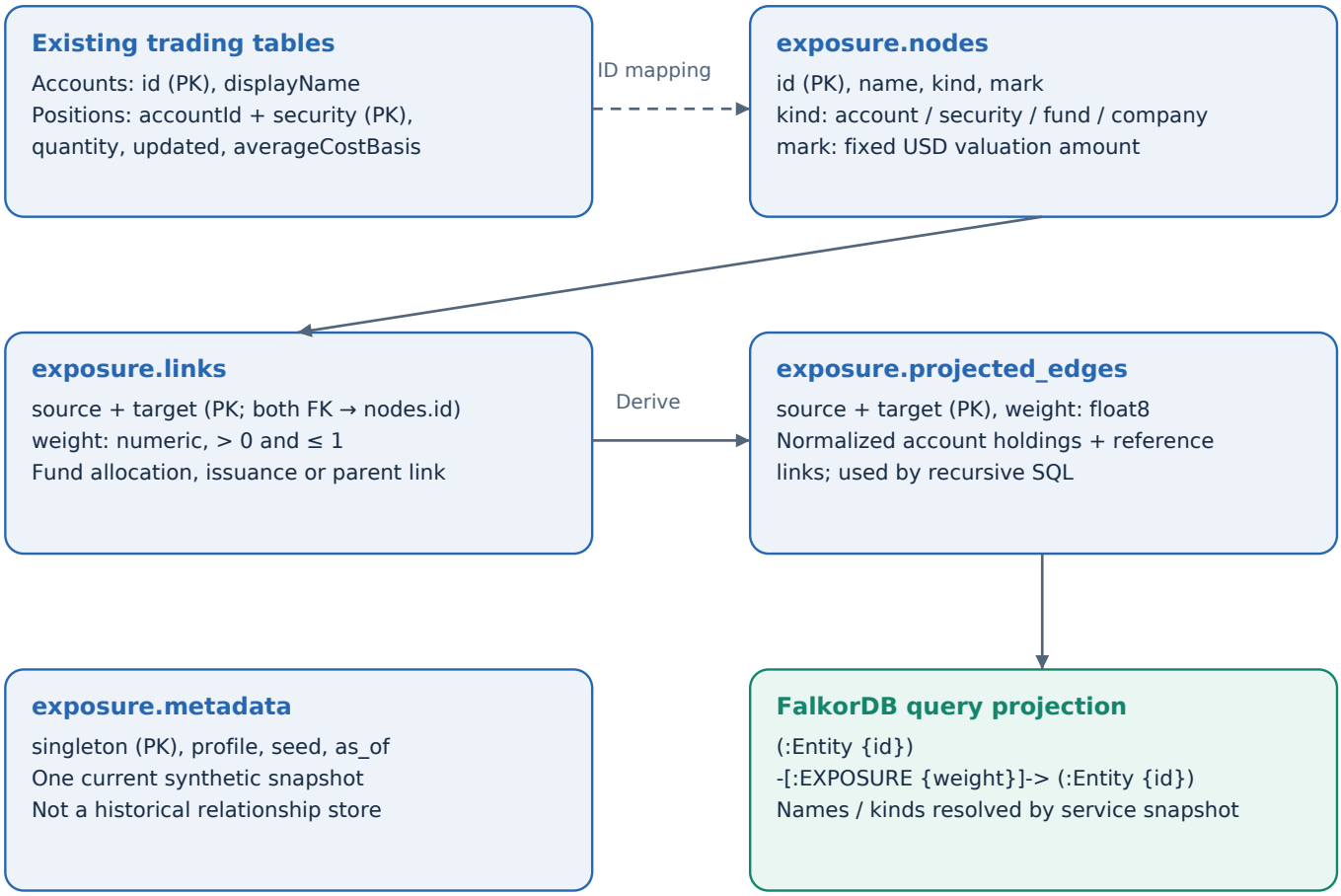
Read-only API: /exposure/api/exposure · UI: /exposure/ · health and Prometheus metrics accompany the service. The maximum supported path depth is 12.

02 / DATA

What data was added

Dataset element	Demo count	Purpose
Accounts	1,000	Different portfolio compositions
Core securities	5,000	Includes 500 funds; multiple securities per issuer
Corporate entities	2,000	Issuer / subsidiary / parent chains
Scenario entities	100	20 isolated known-answer scenarios
Reference links	56,100	Fund allocations, issuance and parent relationships
Account positions	29,460	Current holdings from TraderX Positions

Total: 8,100 entities and 85,560 projected relationships. Funds are included in securities; positions form account-to-security relationships. The larger 100,000-account profile is available but was not measured in this study.



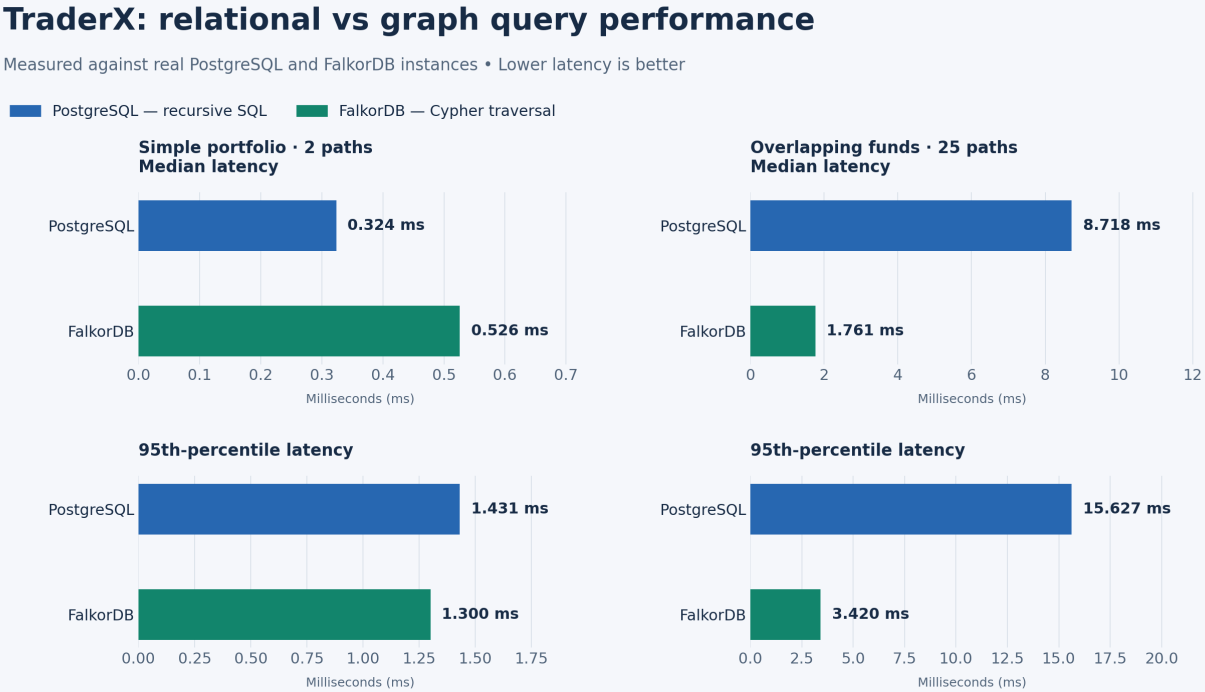
How exposure is calculated

Account edge = quantity × fixed mark / portfolio value. Fund weights multiply along the path; a 40% fund allocation × 60% nested allocation × 50% security holding contributes **12%** of the portfolio. Issuance and corporate-parent links have weight 1: they group exposure rather than model ownership-percentage economics.

Account IDs map to analytical nodes by identifier; this is a derived mapping, not a new foreign key in the existing trading schema. The exposure tables hold one current snapshot, with its as-of time in metadata.

03 / RESULTS

Query latency: measured comparison



Query / metric	PostgreSQL	FalkorDB
Simple: median / p95 / p99	0.324 / 1.431 / 3.832 ms	0.526 / 1.300 / 1.650 ms
Overlapping: median / p95 / p99	8.718 / 15.627 / 17.494 ms	1.761 / 3.420 / 6.594 ms

Method

Each backend ran ten warmup requests followed by 100 timed sequential requests for each query. Both clients reused established database connections. The timer includes client round trip, traversal and explanation-path formatting. Source reconciliation is outside the timed query interval.

The simple case queried account 70000 → Z0G and returned two paths with 14% exposure. The overlapping case queried account 70020 → E0 and returned 25 paths with approximately 0.252977% exposure. Results agreed between SQL and Cypher within floating-point tolerance.

Environment: one managed Linux x86_64 development host with five visible CPUs; Python 3.12.14, PostgreSQL 16.4, FalkorDB 4.16.0. The benchmark has no concurrent clients, cache-flush experiment or network-distance simulation.

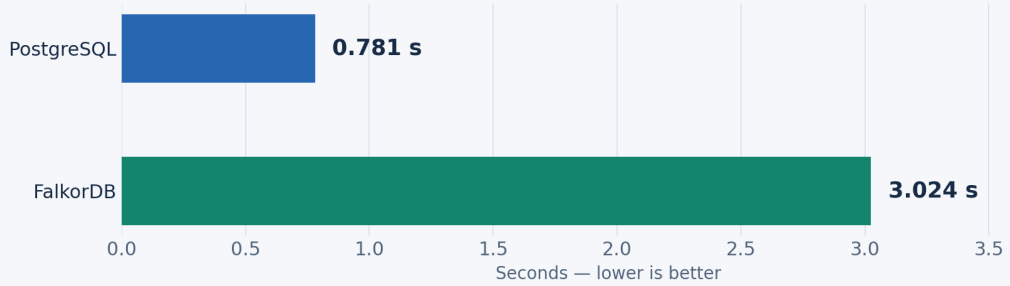
Reading the chart: the panels use different scales. Compare the labeled timings within each query. The 5× result applies to the overlapping-funds median, not to every query or to end-to-end application throughput.

04 / COST AND VALIDATION

Setup cost, correctness and limits

The other side of the comparison: projection setup

Time to create the initial queryable projection from the same source snapshot



FalkorDB took about 3.9× longer to build the initial projection in this implementation.

Snapshot read + projection creation; not full application startup, and not a steady-state query latency measurement.
8,100 entities • 85,560 relationships • One initial build/backend • Measured Nov 12, 2026 • Local development host

What was tested

• Eight real PostgreSQL/FalkorDB integration test methods, including 20 known-answer scenarios, additional portfolio parity, position and allocation mutations, and changed portfolio denominators.

• Non-destructive reseeding; conflicting-profile rejection; invalid weights, cycles and unsupported depth; graph loss and endpoint failure/recovery; HTTP errors and injection-shaped input.

• Browser checks for totals, explanation paths, zero exposure, error display and mobile rendering on both backends. The exposure image and inherited Angular production frontend were built.

• Generated Compose exposure stacks, live position updates, and recovery after an actual FalkorDB container restart. Clean sequential generation, lineage, lifecycle dry-runs, specification, readiness, coverage and frontmatter checks passed.

Boundaries of the evidence

Initial setup is one observed projection build per backend, not full application startup or repeated update-lag statistics. Runtime container checks used stock PostgreSQL 16.4 with the inherited initialization schema; the complete C2 remediation-image stack and full trading end-to-end regression were not exercised.

The model is current-snapshot, long-only and single-currency with fixed USD marks, one exposure-service instance per database and full-snapshot reconciliation. Historical as-of queries, production CDC, multi-replica coordination, derivatives, shorts and the 100,000-account capacity profile remain follow-up work. A CI workflow was added, but hosted CI had not run at the time of this report.

05 / INTEGRATION IMPACT

What changed in core TraderX?

The extension reuses the trading model and adds an analytical capability beside it. Existing Java/.NET trading-service business logic and existing trading-table column definitions were not rewritten. The state generator composes the new capability into descendant runtimes; earlier states retain their own behavior.

Area	Change made
Trading data / schema	Seed synthetic rows into existing Accounts, AccountUsers and Positions; user01 receives synthetic-account entitlements. Advance accounts_seq beyond seeded IDs. Existing columns remain intact. Add a separate exposure schema and SQL projection table.
Reference data and prices	Append synthetic instruments to the generated reference-data CSV, extend the supported ticker lists and add synthetic $100 opening/closing price seeds. Exposure calculation uses fixed marks stored in exposure.nodes, rather than live ticks.
Application navigation / routing	Add a Portfolio Exposure link to the generated Angular header and an NGINX /exposure/ route. The explanation view is served by the new sidecar; existing trading pages remain inherited.
Runtime and observability	Add a Python exposure service; state 018 also adds a persistent, internal FalkorDB service. Add a Prometheus scrape target. Use state-specific Compose projects and database volumes.
Shared generation / tooling	Extend the state catalog, API explorer contract registration, runtime image policy, lifecycle harness, CI and lineage allowlists, and lifecycle validation. Remove obsolete intermediate Compose snapshots from the new generated outputs.
Documentation / tests	Add both specification packs, architecture models, learning contracts and guides, integration/browser tests, benchmark scripts, result artifacts and CI coverage.

Reproducibility and provenance

Local implementation snapshot: **a844b10**. State IDs 017 and 018 were selected after checking open PRs; 016 was already used by PR #470. The report describes a locally implemented and tested experiment; it does not claim an upstream merge or published PR.

Source of numbers: benchmark.json and scripts/benchmark-exposure.py. Full source revision: `30671326a7826b39dbb25eea80d9a45a4de2bdb62cf162e1ac74001ac3b3d458`. This identifies the normalized source snapshot, not a Git commit.

Project: github.com/finos/traderX
Companion proposal: TraderX + FalkorDB proposal

Practical conclusion: the experiment demonstrates useful, explainable portfolio look-through and a graph advantage for the selected overlapping traversal. Broader workload and scale testing should determine where that advantage persists and whether it justifies synchronization and operating costs.